

# CRM Project Documentation

## Overview

This project is a private CRM application built for Developer Tech LLP. It is implemented with a modern full-stack Next.js architecture and supports admin-only login, client management, project tracking, invoice generation, website monitoring, and Docker-based deployment.

## Tech Stack

- Frontend / Backend: Next.js 16 (App Router)
- Database: PostgreSQL 16
- ORM: Prisma 7
- Authentication: NextAuth.js
- Styling: Tailwind CSS 4
- PDF generation: jsPDF + jsPDF-AutoTable
- Deployment: Docker Compose + Nginx
- Utilities: bcryptjs, node-cron, react-hot-toast, next-themes

## Project Structure

- `docker/`
  - `docker-compose.yml` - service definitions for PostgreSQL, app, migrate, nginx
  - `Dockerfile` - multi-stage build for dependencies, build, migration, and production runtime
  - `nginx/nginx.conf` - reverse proxy configuration with security headers and rate limiting
- `prisma/`
  - `schema.prisma` - database schema definition and Prisma datasource
  - `seed.ts` - admin user seed script
- `public/` - static assets
- `scripts/`
  - `backup.sh` - database backup script for Docker and local environments
- `src/`
  - `lib/` - shared utilities and infrastructure
  - `app/` - Next.js app routes and pages
  - `components/` - reusable UI components and forms
- `.env`, `.env.local` - environment configuration
- `README.md` - quick start and deployment documentation
- `PROJECT_DOCUMENTATION.md` - full reference documentation

## How the Project Was Built

### 1. Initialize the Next.js App

- The project started with a Next.js application using the App Router.
- Pages and layouts were built under `src/app/`.
- `src/app/layout.tsx` contains the global layout for the dashboard and auth pages.

### 2. Set up Authentication

- Used NextAuth.js with `CredentialsProvider` in `src/lib/auth.ts`.
- The app uses email and password credentials stored in PostgreSQL.
- Password hashing is implemented with `bcryptjs`.
- JWT sessions are enabled with a 24-hour expiration.
- Custom auth pages defined in `authOptions.pages` for `signIn` and `error`.

### 3. Configure Database and ORM

- Prisma is configured with a PostgreSQL datasource in `prisma/schema.prisma`.
- `src/lib/prisma.ts` creates a singleton Prisma client and supports serverless environments.
- Database models include `User`, `Client`, `Project`, `Invoice`, `Payment`, and `MonitoringLog`.
- Enums represent `ProjectStatus`, `InvoiceStatus`, `MonitorStatus`, and `UserRole`.

### 4. Build Backend API Routes

- API routes are implemented inside `src/app/api/`.
- Authentication routes live under `src/app/api/auth/[...nextauth]/route.ts`.
- CRUD routes exist for clients, invoices, projects, and monitoring.
- Additional endpoints provide dashboard data and authentication status.

### 5. Build UI Components and Pages

- Reusable UI components are stored in `src/components/`.
- Forms for clients, invoices, monitoring, and projects use controlled inputs.
- The dashboard page renders cards and statistics via `src/components/dashboard/StatsCard.tsx`.
- Layout components include `Header`, `Sidebar`, and `DashboardLayout`.

### 6. Add Security and Middleware

- `src/middleware.ts` enforces auth for protected routes and sets security headers.

- The middleware excludes `/api/auth`, `/login`, **static assets**, and **favicon**.
- **Headers include** `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, and `Permissions-Policy`.
- Bot protection and limiting are handled through **Nginx rate limits**.

## 7. Create Docker Deployment

- Docker Compose defines services for:
  - `db`: PostgreSQL database
  - `app`: Next.js application
  - `migrate`: Prisma migration and seed runner
  - `nginx`: reverse proxy
- The `Dockerfile` uses multi-stage build:
  - `deps`: install dependencies and generate Prisma client
  - `builder`: copy code and build Next.js app
  - `migrate`: prepare Prisma CLI for migrations
  - `runner`: run production server via standalone build output
- Nginx is configured to proxy API and app traffic to the Node app and provide cache control for static assets.

## 8. Seed Admin User

- `prisma/seed.ts` creates an admin user when none exists.
- Password is hashed with `bcrypt` before saving to the database.
- Seed uses environment variables: `ADMIN_EMAIL` and `ADMIN_PASSWORD`.

## 9. Add Backup Support

- `scripts/backup.sh` backs up PostgreSQL to a compressed `.sql.gz` file.
- It supports both Docker and local database environments.
- Old backups are deleted after a retention period (default 30 days).

## Environment Variables

| Variable                       | Description                       | Required                                       | Notes                                           |
|--------------------------------|-----------------------------------|------------------------------------------------|-------------------------------------------------|
| <code>POSTGRES_USER</code>     | PostgreSQL username               | Docker only Default: <code>crm_user</code>     |                                                 |
| <code>POSTGRES_PASSWORD</code> | PostgreSQL password               | Docker only Default: <code>crm_password</code> |                                                 |
| <code>POSTGRES_DB</code>       | PostgreSQL database name          | Docker only Default: <code>crm_db</code>       |                                                 |
| <code>DATABASE_URL</code>      | Prisma database connection string | Yes                                            | Used by app and migrations                      |
| <code>NEXTAUTH_SECRET</code>   | Secret for NextAuth JWT           | Yes                                            | Must be kept private                            |
| <code>NEXTAUTH_URL</code>      | Application base URL              | Yes                                            | <code>http://localhost:3000</code> in local dev |
| <code>ADMIN_EMAIL</code>       | Initial admin email               | Yes                                            | Used by seed script                             |
| <code>ADMIN_PASSWORD</code>    | Initial admin password            | Yes                                            | Used by seed script                             |

## Local Development

### Prerequisites

- Node.js 20+
- npm
- PostgreSQL 16

### Setup

1. Clone repository
2. Install dependencies:

```
npm install
```

3. Copy environment example:

```
cp .env.example .env
```

4. Update `.env` with database and auth credentials
5. Generate Prisma client:

```
npx prisma generate
```

6. Apply migrations:

```
npx prisma migrate dev --name init
```

7. Seed admin user:

```
npx prisma db seed
```

8. Start app:

```
npm run dev
```

### App URL

- `http://localhost:3000`

## Docker Deployment

Start all services

```
docker compose -f docker/docker-compose.yml up -d
```

Run database commands

```
docker compose -f docker/docker-compose.yml run --rm migrate db push
docker compose -f docker/docker-compose.yml run --rm migrate db seed
```

Stop services

```
docker compose -f docker/docker-compose.yml down
```

Nginx Reverse Proxy

- Nginx listens on ports 80 and 443
- API routes and app pages are proxied to app:3000
- Rate limiting is enabled for auth and API paths
- SSL configuration lines are present but commented; enable them with valid certificates

Database Schema Summary

User

- id: string, cuid
- email: unique
- password: hashed string
- name: default "Admin"
- role: enum ADMIN
- createdAt, updatedAt

Client

- id, name, phone, email, company, address, projectType, notes
- relations to Project and Invoice

Project

- title, description, status, dueDate, progress
- clientId relation to Client

Invoice

- invoiceNumber, serviceName, amount, date, status
- clientId relation to Client
- payments relation to Payment

Payment

- amount, date, method, invoiceId

MonitoringLog

- url, status, statusCode, lastChecked, responseTime

Authentication Flow

1. User enters email and password on login page.
2. NextAuth uses credentials provider in src/lib/auth.ts.
3. authorize() checks the database for a matching email.
4. Password is compared via bcrypt.compare().
5. On success, the user object is returned and JWT session is created.
6. Middleware in src/middleware.ts protects private routes and applies security headers.

Security Practices

- Password hashing with bcrypt
- JWT session strategy
- Custom NextAuth sign-in and error pages
- CSRF protection via NextAuth
- Security headers from middleware.ts
- Nginx request limiting and hidden file blocking
- Environment variables excluded from Git via .gitignore

Commands Reference

| Command                            | Description                        |
|------------------------------------|------------------------------------|
| npm install                        | Install dependencies               |
| npm run dev                        | Start development server           |
| npm run build                      | Build Next.js app for production   |
| npm run start                      | Start production server            |
| npx prisma generate                | Generate Prisma client             |
| npx prisma migrate dev --name init | Create and apply schema migrations |
| npx prisma db seed                 | Seed initial admin user            |

|                                                                                   |                                  |
|-----------------------------------------------------------------------------------|----------------------------------|
| <code>docker compose -f docker/docker-compose.yml up -d</code>                    | Start Docker services            |
| <code>docker compose -f docker/docker-compose.yml down</code>                     | Stop Docker services             |
| <code>docker compose -f docker/docker-compose.yml run --rm migrate db push</code> | Apply Prisma schema in container |
| <code>docker compose -f docker/docker-compose.yml run --rm migrate db seed</code> | Seed admin in container          |

## Production Notes

- Use `NEXTAUTH_URL` with your actual domain
- Use a strong `NEXTAUTH_SECRET`
- Configure SSL certificates and enable HTTPS in Nginx
- Ensure host ports 80, 443, 3000, and 5432 are available
- Use `docker compose` on the host server for reliable deployment
- Set a cron job for `scripts/backup.sh` to preserve database backups

## How to Extend the App

- Add more user roles by extending `UserRole` enum and auth logic
- Add email notifications for invoices or status updates
- Expand monitoring with scheduled URL checks and alerts
- Add public client portals with role-based access
- Improve analytics in the dashboard with charts and trends

## Additional Notes

- `.env*` files are ignored by Git, so secrets remain local.
- Prisma migrations are managed in `prisma/migrations` but ignored for production builds in `.gitignore`.
- The app is currently admin-only; no public registration flow exists.

---

## Generated Documentation

This file is the complete technical documentation for the CRM project and can be used as a reference for development, deployment, and maintenance.